

Smart Clinic Management System

BIT1123 – Object Oriented Programming (Java)

Final Project Report

Group Members

1. Elmenoufy Mohamed Ezat Abdelhamid / 202401010544
2. .Rashid Mohamed Omer / 202501010714
3. WALEID FADLLAH ELDIRDERY AHMED /202501010472
4. Adil Elkhedir Ahmed Elkhedir /202505010496
5. ABDELRHMAN ABDELRHMAN AHMED IBRAHEEM /202505010324

Lecturer:NAZMIRUL IZZAD BIN NASIR

Academic Year: 2025/2026

Table of Contents

1. Introduction
2. Project Overview
3. Sustainable Development Goal
4. Project Objectives
5. System Requirements and Main Features
6. System Design and Architecture
7. Object-Oriented Programming Concepts
8. Collections and File I/O
9. Graphical User Interface
10. Processing and Business Logic
11. UML Class Design
12. Testing and Sample Outputs
13. Code Quality and Project Organization
14. Limitations and Future Improvements
15. Conclusion
16. References

1. Introduction

Smart Clinic Management System is a Java desktop application developed for the BIT1123 Object Oriented Programming (Java) final project. The system models common clinic activities through a graphical user interface and demonstrates the use of object-oriented programming principles in a practical, real-world context.

The application focuses on managing patients, doctors, appointments, and prescriptions. Clinic data is stored in CSV files so that information can be loaded when the application starts and saved after changes. The project therefore combines GUI development, collections, file handling, validation, and OOP concepts in one integrated system.

2. Project Overview

The system provides a centralized way to manage basic clinic information. Patients and doctors are represented as objects, appointments connect patients with doctors, and prescriptions are associated with appointments. The ClinicManager class handles the main business operations, while FileManager handles persistent CSV storage and MainFrame provides the graphical interface.

Main entities:

- Person – abstract parent class containing common personal information and behavior.
- Patient – represents a clinic patient and stores age, gender, phone number, and condition.
- Doctor – represents a doctor and stores contact information and specialty.
- Appointment – represents a scheduled meeting between a patient and a doctor.
- Prescription – represents medication information associated with an appointment.

3. Sustainable Development Goal

This project is aligned with SDG 3 – Good Health and Well-being.

The project supports the goal by providing an organized digital system for managing information related to patients, doctors, appointments, and prescriptions. By keeping clinic information in structured records and making it accessible through a single interface, the system demonstrates how software can support more organized healthcare administration.

The application is an educational software project and is not intended to replace professional medical systems or provide medical diagnosis.

4. Project Objectives

- Develop a functional Java GUI application for a clinic scenario.
- Apply encapsulation, inheritance, abstraction, and polymorphism.
- Use collections to manage multiple clinic objects.
- Use file I/O to provide persistent storage through CSV files.
- Implement validation and business rules for clinic operations.

- Provide a clear and organized project structure suitable for GitHub.
- Demonstrate a real-world application rather than a CRUD-only example.

5. System Requirements and Main Features

The main functions implemented in the system include:

- Adding patients.
- Adding doctors.
- Creating appointments between existing patients and doctors.
- Creating prescriptions for existing appointments.
- Deleting patients, doctors, appointments, and prescriptions.
- Displaying clinic information and statistics through the GUI.
- Validating relationships such as patient/doctor IDs before creating appointments.
- Automatically generating identifiers such as PT-001, DR-001, AP-001, and PR-001.
- Saving and loading data using CSV files.

6. System Design and Architecture

The project is organized into packages according to responsibility. This separation improves maintainability and makes the application easier to understand.

- clinic.model – contains the core data classes: Person, Patient, Doctor, Appointment, and Prescription.
- clinic.service – contains ClinicManager for business logic and FileManager for CSV file operations.
- clinic.ui – contains MainFrame, which provides the graphical user interface.
- clinic – contains Main.java, which starts the application.
- data – contains patients.csv, doctors.csv, appointments.csv, and prescriptions.csv.

Project structure:

- src/clinic/Main.java
- src/clinic/model/Person.java
- src/clinic/model/Patient.java
- src/clinic/model/Doctor.java
- src/clinic/model/Appointment.java
- src/clinic/model/Prescription.java
- src/clinic/service/ClinicManager.java
- src/clinic/service/FileManager.java
- src/clinic/ui/MainFrame.java
- data/patients.csv
- data/doctors.csv
- data/appointments.csv
- data/prescriptions.csv

7. Object-Oriented Programming Concepts

7.1 Encapsulation

The model classes encapsulate their data using fields and public getter/setter methods. For example, Patient stores age, gender, and condition as private attributes and provides methods such as `getAge()`, `setAge()`, `getGender()`, `setGender()`, `getCondition()`, and `setCondition()`. This prevents direct uncontrolled access to object data.

7.2 Inheritance

Patient and Doctor inherit from the abstract Person class. Person stores common attributes such as id, name, and phone, while the subclasses add their own specialized information.

7.3 Abstraction

Person is declared as an abstract class and defines abstract methods such as `getRole()` and `displayInfo()`. This provides a common contract for the different types of people in the clinic.

7.4 Polymorphism

The system maintains a `List<Person>` called `people`. Patient and Doctor objects are stored in this list through the superclass type Person. When `displayInfo()` is called through a Person reference, Java uses the overridden implementation belonging to the actual object at runtime. This demonstrates runtime polymorphism using a superclass reference.

Example:

```
for (Person person : people) {  
    info.add(person.displayInfo());  
}
```

8. Collections and File I/O

8.1 Collections

ClinicManager uses `LinkedHashMap` collections for patients, doctors, appointments, and prescriptions. A `List<Person>` is also used to demonstrate superclass-based storage and polymorphism.

8.2 File I/O

FileManager is responsible for reading and writing CSV files. The data folder contains four files: `patients.csv`, `doctors.csv`, `appointments.csv`, and `prescriptions.csv`. This allows the system to preserve data between application runs.

The application uses the pipe character (`|`) as the delimiter in the CSV data files.

9. Graphical User Interface

The application uses Java Swing to provide a desktop graphical user interface. MainFrame is the main GUI class. The interface allows users to view and manage the main clinic entities without interacting directly with the CSV files.

Screenshots of the final GUI should be inserted in this section before submission.

- [Insert Screenshot 1: Main Dashboard]
- [Insert Screenshot 2: Patient Management]
- [Insert Screenshot 3: Doctor Management]
- [Insert Screenshot 4: Appointment Management]
- [Insert Screenshot 5: Prescription Management]

10. Processing and Business Logic

The application includes processing beyond simple data entry. ClinicManager validates related records before creating appointments and prescriptions. For example, an appointment cannot be created unless the referenced patient and doctor IDs exist. Similarly, a prescription cannot be created unless the referenced appointment exists.

The manager also generates sequential IDs, refreshes counters from existing data, removes dependent records when a patient or doctor is deleted, and calculates basic clinic statistics.

11. UML Class Design

The UML design represents the relationships between the main classes. Person is the abstract superclass for Patient and Doctor. ClinicManager coordinates the model objects and FileManager handles persistence. MainFrame uses the manager to provide the user interface.

Insert the final UML Class Diagram here. The diagram should match the final source code and show class names, important attributes/methods, inheritance, and major associations.

[Insert Final UML Class Diagram Here]

12. Testing and Sample Outputs

Testing should be performed before final submission. The following test cases cover the main operations and validation rules of the application.

Test Case	Input / Action	Expected Result
Add Patient	Enter valid patient information.	A new patient is created with a generated ID.
Add Doctor	Enter valid doctor information.	A new doctor is created with a generated ID.
Create Appointment	Use an existing patient ID and doctor ID.	A scheduled appointment is created.
Invalid Patient ID	Create an appointment with a	The operation is rejected with

	non-existing patient ID.	a validation message.
Invalid Doctor ID	Create an appointment with a non-existing doctor ID.	The operation is rejected with a validation message.
Create Prescription	Use an existing appointment ID.	A prescription is created.
Invalid Appointment ID	Create a prescription with a non-existing appointment ID.	The operation is rejected.
Delete Patient	Delete an existing patient.	The patient is removed and related appointments are removed.
File Persistence	Restart the application after saving data.	Previously saved records are loaded from CSV files.
Polymorphism	Store Patient and Doctor objects in List<Person> and call displayInfo().	The overridden method for the actual object type is executed.

Insert screenshots of successful test results and GUI outputs below:

- [Insert Test Screenshot 1]
- [Insert Test Screenshot 2]
- [Insert Test Screenshot 3]

13. Code Quality and Project Organization

The project separates the model, service, and UI responsibilities into different packages. Class and method names describe their responsibilities, and the code uses access modifiers and object-oriented relationships to organize the application.

The GitHub repository should contain the source code, data files, README.md, UML documentation, and .gitignore. Compiled output and IDE-specific files should be excluded through .gitignore.

14. Limitations and Future Improvements

- The current system is a desktop educational project and does not provide a secure multi-user backend.
- CSV storage is suitable for the project but a database would be more appropriate for a larger clinic system.
- Authentication and role-based access control could be added in a future version.
- More advanced appointment conflict checking could be implemented.
- Search, filtering, and reporting features could be expanded.
- The GUI could be improved with additional usability and accessibility features.

15. Conclusion

The Smart Clinic Management System demonstrates how Java Object-Oriented Programming can be applied to a realistic healthcare management scenario. The system combines a graphical interface with collections, file persistence, validation, and business logic for patients, doctors, appointments, and prescriptions.

The project demonstrates encapsulation, inheritance, abstraction, and runtime polymorphism while supporting SDG 3 – Good Health and Well-being. The modular package structure also makes the system easier to maintain and extend.

16. References

1. BIT1123 Object Oriented Programming (Java) – Final Project Brief, 2025/2026.
2. Oracle Java Documentation – Java language and standard library documentation.
3. Project source code and UML documentation in the SmartClinicProject1 GitHub repository.
4. United Nations – Sustainable Development Goal 3: Good Health and Well-being.

17. github link

<https://github.com/medoelmenofy6-bot/final-OOP>